

ADAMS DATABASE PROJECT

50 Panel Q&A Reviewer - Taglish Explanation + Visual Memory Guide

Purpose: This reviewer is for practice only. It is intentionally beginner-friendly and includes explanation lines so you can defend ADAMS as an Information Management / DBMS project.

Color / Keyword Guide	Meaning
Blue keywords	DBMS concept or technical term.
Green keywords	How the concept appears in ADAMS.
Orange/Memory cue	Simple picture or mental cue to remember the answer.
Red line	The one sentence you should memorize.
Defense Answer Formula	Definition + ADAMS example. Example: A primary key uniquely identifies each record. In ADAMS, students.student_id uniquely identifies each student.
Final user roles	Superadmin and Staff only. Student/applicant is treated as an admission record, not a final login role.

A. DBMS and MySQL Foundations

Q1. What is the main focus of your project?

Answer: The main focus of ADAMS is to manage admission documents and applicant/student records using a relational database.

Defense line: ADAMS helps the office record, search, monitor, print, and archive admission records through a database-driven system.

Must remember: ADAMS = admission records + document requirements + database tracking.

Q2. Why is ADAMS considered a database project?

Answer: It is considered a database project because the main features depend on tables, keys, relationships, and SQL queries.

Defense line: The system is not just a form; it depends on tables such as students, student_requirements, requirement_types, users, logs, and archived_students.

Must remember: Always connect your answer to tables and relationships.

Q3. What is a database?

Answer: A database is an organized collection of related data that can be stored, searched, updated, and managed.

Defense line: In ADAMS, the database stores applicant records, requirement status, user accounts, logs, and archived student records.

Must remember: Database = organized data.

Q4. What is a DBMS?

Answer: A DBMS is software used to create, manage, secure, and retrieve records from a database.

Defense line: MySQL/MariaDB is the DBMS used to manage the ADAMS database.

Must remember: DBMS = software that manages the database.

Q5. Why did you use MySQL?

Answer: MySQL is suitable because it supports relational tables, primary keys, foreign keys, joins, constraints, and SQL reporting.

Defense line: Because ADAMS uses related data, MySQL is appropriate for managing structured admission records.

Must remember: MySQL fits relational data.

Q6. What type of DBMS is MySQL?

Answer: MySQL is an RDBMS, or Relational Database Management System.

Defense line: It stores data in related tables, which matches the design of ADAMS.

Must remember: MySQL = RDBMS.

Q7. What is a relational database?

Answer: A relational database stores data in tables and connects those tables using keys.

Defense line: For example, students are connected to student_categories, and requirements are connected through student_requirements.

Must remember: Relational = tables connected by keys.

Q8. Why is a relational database suitable for ADAMS?

Answer: ADAMS has many related records: students, categories, requirements, uploaded files, users, logs, and archives.

Defense line: A relational design prevents scattered data and allows accurate tracking of requirements per student.

Must remember: ADAMS data is connected, so relational database is appropriate.

Q9. What is SQL?

Answer: SQL means Structured Query Language. It is used to retrieve, insert, update, delete, and manage database data.

Defense line: We use SQL to pull student records, check requirements, count records, and generate monitoring reports.

Must remember: SQL is the language of the database.

Q10. What are the main SQL command categories?

Answer: The main categories are DDL, DML, DQL, DCL, and TCL.

Defense line: For our project defense, the most common commands are CREATE, SELECT, INSERT, UPDATE, DELETE, and JOIN.

Must remember: DDL = structure, DML = data changes, DQL = SELECT.

B. SQL Command Categories and Database Objects

Q11. What is DDL?

Answer: DDL means Data Definition Language. It defines or changes database structures.

Defense line: Examples are CREATE TABLE, ALTER TABLE, and DROP TABLE.

Must remember: DDL changes structure.

Q12. What is DML?

Answer: DML means Data Manipulation Language. It changes the records inside tables.

Defense line: Examples are INSERT, UPDATE, and DELETE.

Must remember: DML changes data.

Q13. What is DQL?

Answer: DQL means Data Query Language. It retrieves records from the database.

Defense line: The main DQL command is SELECT.

Must remember: DQL retrieves data.

Q14. What is a table?

Answer: A table stores related data in rows and columns.

Defense line: For example, the students table stores applicant/student information.

Must remember: Table = container of related records.

Q15. What is a field or column?

Answer: A field or column is one data attribute in a table.

Defense line: For example, first_name, last_name, email, and category_id are fields in the students table.

Must remember: Column = type of information.

Q16. What is a record or row?

Answer: A record or row is one complete entry in a table.

Defense line: One applicant in the students table is one record.

Must remember: Row = one complete item/person/transaction.

Q17. What is a primary key?

Answer: A primary key uniquely identifies each record in a table.

Defense line: In ADAMS, students.student_id uniquely identifies each student record.

Must remember: Primary key = unique ID.

Q18. What is a foreign key?

Answer: A foreign key links one table to another table.

Defense line: In ADAMS, students.category_id links a student to student_categories.category_id.

Must remember: Foreign key = connection key.

Q19. What is a unique key?

Answer: A unique key prevents duplicate values in a field or combination of fields.

Defense line: For example, users.username must be unique so two users cannot use the same username.

Must remember: Unique key = no duplicate value.

Q20. What is referential integrity?

Answer: Referential integrity keeps table relationships valid and consistent.

Defense line: For example, a student category assigned to a student should exist in the student_categories table.

Must remember: Referential integrity protects relationships.

C. Business Rules, Relationships, and Cardinality

Q21. What are business rules in database design?

Answer: Business rules describe how records should relate based on the actual process of the organization.

Defense line: In ADAMS, one business rule is that a student belongs to one category when classified.

Must remember: Business rules become relationships.

Q22. Give one business rule in ADAMS.

Answer: Each student/applicant belongs to one student category when classified.

Defense line: This is shown by the relationship between student_categories and students.

Must remember: Safe example: one category can have many students.

Q23. Why do different student categories need different requirements?

Answer: Different admission categories may have different document checklists.

Defense line: For example, a transferee may need records that are different from a freshman applicant.

Must remember: Different category = different checklist.

Q24. What table handles category-based requirements?

Answer: The category_requirements table handles the requirements assigned to each category.

Defense line: It connects student_categories and requirement_types.

Must remember: category_requirements = checklist setup table.

Q25. Why do you need category_requirements?

Answer: It avoids repeating requirement lists and supports a many-to-many relationship between categories and requirement types.

Defense line: One category can have many requirements, and one requirement can be required by many categories.

Must remember: Bridge table prevents repetition.

Q26. What is a bridge table?

Answer: A bridge table connects two tables that have a many-to-many relationship.

Defense line: In ADAMS, category_requirements is the bridge table between student_categories and requirement_types.

Must remember: Bridge table = connector table.

Q27. What is cardinality?

Answer: Cardinality describes how many records in one table can relate to records in another table.

Defense line: Examples are one-to-many and many-to-many relationships in the ERD.

Must remember: Cardinality = how many records are connected.

Q28. Give one example of a one-to-many relationship in ADAMS.

Answer: One student category can classify many students.

Defense line: student_categories to students is a one-to-many relationship.

Must remember: 1 category, many students.

Q29. Give one example of a many-to-many relationship in ADAMS.

Answer: Student categories and requirement types are many-to-many.

Defense line: This is implemented using category_requirements.

Must remember: Many categories, many requirements.

Q30. What table stores the actual student requirement status?

Answer: The student_requirements table stores the actual requirement status of each student.

Defense line: It stores fields such as student_id, requirement_id, file_name, uploaded_by, status, upload_date, and uploaded_at.

Must remember: student_requirements = actual monitoring table.

D. Normalization and Data Anomalies

Q31. What is normalization?

Answer: Normalization is the process of organizing data into separate related tables to reduce redundancy and improve consistency.

Defense line: ADAMS separates students, categories, requirement types, student requirements, users, logs, and archives.

Must remember: Normalization = clean table design.

Q32. Why did you normalize your database?

Answer: We normalized the database to avoid duplicate data, repeating groups, and update errors.

Defense line: Instead of placing all requirements in one student row, ADAMS stores requirements in separate related tables.

Must remember: Normalization avoids redundancy.

Q33. What is UNF?

Answer: UNF means Unnormalized Form. It contains repeating groups or non-atomic values.

Defense line: An example is one row containing Requirement1, Status1, Requirement2, and Status2.

Must remember: UNF = messy/manual logbook style.

Q34. How did ADAMS apply 1NF?

Answer: ADAMS applied 1NF by removing repeating groups and making values atomic.

Defense line: Each requirement is stored as a separate record instead of many requirement columns in one row.

Must remember: 1NF = one value per field.

Q35. How did ADAMS apply 2NF?

Answer: ADAMS applied 2NF by separating data based on its purpose.

Defense line: Student data, category data, requirement type data, and requirement status data are placed in separate tables.

Must remember: 2NF = separate tables by purpose.

Q36. How did ADAMS apply 3NF?

Answer: ADAMS applied 3NF by separating supporting data such as users, activity logs, print logs, password reset tokens, documents, and archived records.

Defense line: This avoids storing user, print, and archive details repeatedly inside the students table.

Must remember: 3NF = remove indirect dependencies.

Q37. What problem happens if you use only one big table?

Answer: One big table causes duplicated data, hard updates, missing records, and accidental deletion of important information.

Defense line: For ADAMS, this would make requirement tracking confusing and inconsistent.

Must remember: One big table = redundancy and anomalies.

Q38. What is an update anomaly?

Answer: An update anomaly happens when the same data appears in multiple places and not all copies are updated.

Defense line: Example: changing a requirement name in many repeated rows.

Must remember: Update anomaly = inconsistent edits.

Q39. What is an insertion anomaly?

Answer: An insertion anomaly happens when you cannot add one type of data without another unrelated data.

Defense line: Example: unable to add a new requirement type unless a student record already exists.

Must remember: Insertion anomaly = cannot insert independently.

Q40. What is a deletion anomaly?

Answer: A deletion anomaly happens when deleting a record also removes important related information by accident.

Defense line: Example: deleting a student accidentally losing the only record of a requirement if the database is not normalized.

Must remember: Deletion anomaly = accidental data loss.

E. ERD, EERD, SQL Checking, and SQL vs NoSQL

Q41. What is an ERD?

Answer: An ERD, or Entity Relationship Diagram, shows the database entities/tables, attributes, keys, and relationships.

Defense line: Our ERD shows tables such as students, users, requirement_types, student_requirements, documents, logs, and archives.

Must remember: ERD = database map.

Q42. What is an EERD?

Answer: An EERD, or Enhanced Entity Relationship Diagram, extends ERD by showing enhanced concepts like specialization and generalization.

Defense line: In ADAMS, it shows USER specialization and student classification.

Must remember: EERD = enhanced ERD.

Q43. What is the difference between ERD and EERD?

Answer: ERD shows basic entities and relationships, while EERD shows enhanced modeling concepts such as subtypes.

Defense line: Our EERD explains that system users are specialized into Superadmin and Staff.

Must remember: ERD = basic; EERD = enhanced.

Q44. What is specialization in EERD?

Answer: Specialization means dividing a general entity into more specific subtypes.

Defense line: In ADAMS, USER is specialized into SUPERADMIN_USER and STAFF_USER only.

Must remember: Specialization = general to specific.

Q45. Who are the system users in ADAMS?

Answer: The final system users are Superadmin and Staff only.

Defense line: Superadmin manages system-wide accounts, settings, categories, requirements, logs, and reports. Staff handles routine searching, updating, uploading, monitoring, printing, and archiving subject to granted access.

Must remember: Applicants/students are records, not final login roles.

Q46. What is the purpose of the data dictionary?

Answer: The data dictionary explains every table field, data type, key/constraint, and meaning.

Defense line: It helps verify that the ERD and SQL schema are consistent.

Must remember: Data dictionary = table field guide.

Q47. What SQL command will you use to view all tables?

Answer: Use SHOW TABLES;

Defense line: This lets us confirm all ADAMS tables are available in the selected database.

Must remember: SHOW TABLES = list tables.

Q48. What SQL command will you use to check table structure?

Answer: Use DESCRIBE table_name;

Defense line: Example: DESCRIBE students; or DESCRIBE student_requirements;

Must remember: DESCRIBE = check columns.

Q49. What SQL command will you use to retrieve student records?

Answer: Use SELECT. Example: SELECT * FROM students;

Defense line: For cleaner output, we can select specific columns such as student_id, student_name, email, and category_id.

Must remember: SELECT = retrieve data.

Q50. Why did you choose SQL/MySQL instead of NoSQL?

Answer: We chose SQL/MySQL because ADAMS uses structured and related data that needs keys, joins, constraints, and accurate reporting.

Defense line: NoSQL is useful for flexible/unstructured data, but ADAMS is better represented as relational tables.

Must remember: Relational data = SQL/MySQL.

Bonus: Quick SQL Checking Commands

How to use: These are quick commands for practice or demo. They are generic starter commands on purpose, but they use the correct ADAMS table and column names.

Select database and show tables

```
USE doc_admission_db;
SHOW TABLES;
```

Check table structure

```
DESCRIBE users;
DESCRIBE students;
DESCRIBE student_requirements;
DESCRIBE documents;
```

Retrieve all students

```
SELECT *
FROM students;
```

View students with categories

```
SELECT s.student_id, s.student_name, c.category_name
FROM students s
LEFT JOIN student_categories c
    ON c.category_id = s.category_id
ORDER BY s.student_name;
```

View requirements per category

```
SELECT c.category_name, r.requirement_name
FROM category_requirements cr
JOIN student_categories c
    ON c.category_id = cr.category_id
JOIN requirement_types r
    ON r.requirement_id = cr.requirement_id
ORDER BY c.category_name, r.requirement_name;
```

Check missing or pending requirements

```
SELECT s.student_id, s.student_name, c.category_name,
       r.requirement_name,
       COALESCE(sr.status, 'MISSING') AS status_result
FROM students s
JOIN student_categories c
    ON c.category_id = s.category_id
JOIN category_requirements cr
    ON cr.category_id = s.category_id
JOIN requirement_types r
    ON r.requirement_id = cr.requirement_id
LEFT JOIN student_requirements sr
    ON sr.student_id = CAST(s.student_id AS CHAR)
   AND sr.requirement_id = r.requirement_id
WHERE sr.student_requirement_id IS NULL
     OR sr.status IS NULL
     OR sr.status IN ('PENDING', 'INSUFFICIENT', 'REJECTED')
ORDER BY s.student_name, r.requirement_name;
```

Count records per major table

```
SELECT 'students' AS table_name, COUNT(*) AS total FROM students
UNION ALL
SELECT 'student_categories', COUNT(*) FROM student_categories
UNION ALL
SELECT 'requirement_types', COUNT(*) FROM requirement_types
UNION ALL
SELECT 'student_requirements', COUNT(*) FROM student_requirements
UNION ALL
SELECT 'users', COUNT(*) FROM users
UNION ALL
SELECT 'activity_log', COUNT(*) FROM activity_log
UNION ALL
SELECT 'print_logs', COUNT(*) FROM print_logs
UNION ALL
SELECT 'archived_students', COUNT(*) FROM archived_students;
```

Emergency Answer Kapag Hindi Mo Alam ang Sagot

Safe answer: Since our subject is Information Management / DBMS, our focus is the proper application of database design: entities, attributes, keys, relationships, normalization, SQL queries, and data integrity in ADAMS.